

COURSE CODE/TITLE:	CPE 212 Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	09/03/2026	DATE SUBMITTED:	09/03/2026
NAME:	Interno, Jerald San M.	INSTRUCTOR:	Engr. Jimlord

ACTIVITY NO. 6.1 Targeting Specific Nodes

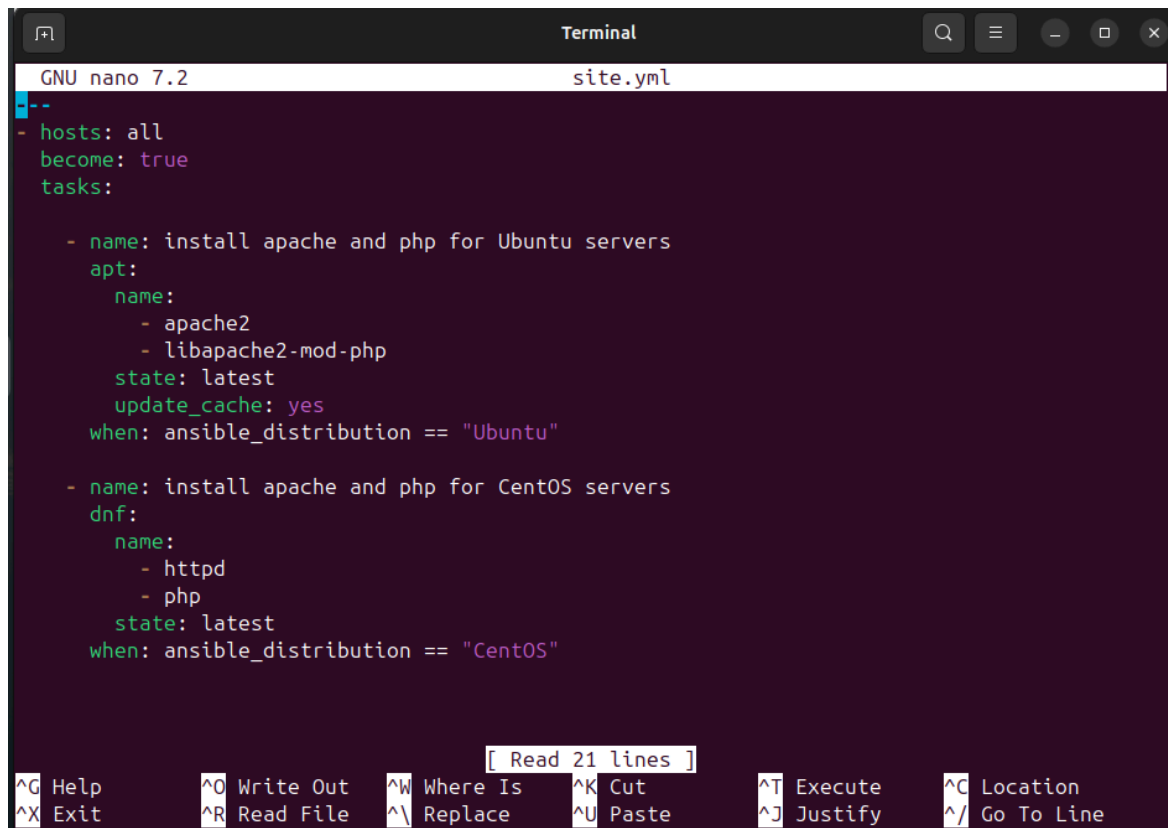
1. PROCEDURE OUTPUT

Task 1: Targeting Specific Nodes

1. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.
2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:
Make sure to save the file and exit.
Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.
3. Edit the site.yml by following the image below:
Make sure to save the file and exit.
The pre-tasks command tells the ansible to run it before any other thing. In the pre-tasks, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at web_servers. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.
Run the site.yml file and describe the result.
4. Let's try to edit again the site.yml file. This time, we are going to add plays targeting the other servers. This time we target the db_servers by adding it on the current site.yml. Below is an example: (Note add this at the end of the playbooks from task 1.3).
Make sure to save the file and exit.
Run the site.yml file and describe the result.
5. Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: `systemctl status mariadb`. Do this on the CentOS server also.
Describe the output.
6. Edit the site.yml again. This time we will append the code to configure installation on the file_servers group. We can add the following on our file.
Make sure to save the file and exit.
Run the site.yml file and describe the result.
The testing of the file_servers is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

1.

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```



```
GNU nano 7.2 site.yml
--
- hosts: all
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
        when: ansible_distribution == "CentOS"
```

[Read 21 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify	^_ Go To Line

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano inventory.
yml
```

2.

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible all -m  
ping -i inventory.yaml -k  
'SSH password:  
192.168.56.103 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "changed": false,  
  "ping": "pong"  
}  
192.168.56.101 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "changed": false,  
  "ping": "pong"  
}  
192.168.56.104 | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python": "/usr/bin/python3"  
  },  
  "changed": false,  
  "ping": "pong"  
}
```

3.

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```

```
GNU nano 7.2                                site.yml *
- hosts: all
  become: true
  pre_tasks:

    - name: install updates (CentOS)
      dnf:
        update_only: yes
        update_cache: yes
      when: ansible_distribution == "CentOS"

    - name: install updates (Ubuntu)
      apt:
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
      when: ansible_distribution == "Ubuntu"
```

```

[ERROR]: user interrupted execution
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook
k -i inventory.yaml site.yml -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****
*****

TASK [Gathering Facts] *****
*****
ok: [192.168.56.103]
ok: [192.168.56.104]
ok: [192.168.56.101]

TASK [install updates (CentOS)] *****
*****
skipping: [192.168.56.103]
skipping: [192.168.56.104]
ok: [192.168.56.101]

TASK [install updates (Ubuntu)] *****
*****
skipping: [192.168.56.101]
changed: [192.168.56.104]
changed: [192.168.56.103]

PLAY [web_servers] *****
*****

TASK [Gathering Facts] *****
*****

```

```
TASK [Gathering Facts] *****
*****
ok: [192.168.56.103]
ok: [192.168.56.104]

TASK [install apache and php for Ubuntu servers] *****
*****
ok: [192.168.56.103]
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
*****
skipping: [192.168.56.103]
skipping: [192.168.56.104]

PLAY RECAP *****
*****
192.168.56.101      : ok=2    changed=0    unreachable=0    failed=0
192.168.56.103      : ok=4    changed=1    unreachable=0    failed=0
192.168.56.104      : ok=4    changed=1    unreachable=0    failed=0
192.168.56.101      : skipped=1    rescued=0    ignored=0
192.168.56.103      : skipped=2    rescued=0    ignored=0
192.168.56.104      : skipped=2    rescued=0    ignored=0
```

4.

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```

```

GNU nano 7.2                                site.yml
- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb server (Ubuntu)
      apt:
        name: mariadb-server
        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: install mariadb server (CentOS)
      dnf:
        name: mariadb-server
        state: latest
        when: ansible_distribution == "CentOS"

jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml site.yml -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [db_servers] *****
TASK [Gathering Facts] *****
ok: [192.168.56.101]

TASK [install mariadb server (Ubuntu)] *****
skipping: [192.168.56.101]

TASK [install mariadb server (CentOS)] *****
changed: [192.168.56.101]

PLAY RECAP *****
192.168.56.101      : ok=2    changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

```

5.

```
jerald@workstation:~/CPE212_Jerald_Interno$ ssh jeraldinterno@192.168.56.101
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Sat Sep  5 21:23:45 2026 from 192.168.56.1
[jeraldinterno@localhost ~]$ systemctl status mariadb
○ mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; vendor preset: enabled)
   Active: inactive (dead)
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/

[jeraldinterno@localhost ~]$ exit
logout
Connection to 192.168.56.101 closed.
```

6.

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```

GNU nano 7.2

site.yml *

```
hosts: file_servers
become: true
tasks:

  - name: install samba package
    package:
      name: samba
      state: latest
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yml site.yml -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]

TASK [install samba package] *****
ok: [192.168.56.103]

PLAY RECAP *****
192.168.56.103      : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the site.yml file. Add tags to the playbook. After the name, we can place the tags: name_of_tag. This is an arbitrary command, which means you can use any name for a tag. Make sure to save the file and exit.

Run the site.yml file and describe the result.

2. On the local machine, try to issue the following commands and describe each result:

2.1 ansible-playbook --list-tags site.yml

2.2 ansible-playbook --tags centos --ask-become-pass site.yml

2.3 ansible-playbook --tags db --ask-become-pass site.yml

2.4 ansible-playbook --tags apache --ask-become-pass site.yml

2.5 ansible-playbook --tags "apache,db" --ask-become-pass site.yml

```
gerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```

```
GNU nano 7.2                                site.yml
---
- hosts: all
  become: true
  pre_tasks:
    - name: install updates (CentOS)
      dnf:
        update_only: yes
        update_cache: yes
      when: ansible_distribution == "CentOS"
      tags: centos
    - name: install updates (Ubuntu)
      apt:
        update_cache: yes
      when: ansible_distribution == "Ubuntu"
      tags: ubuntu
- hosts: web_servers
  become: true
  tasks:
    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
```

```
GNU nano 7.2                                site.yml
tags: [ubuntu, apache]

- name: install apache and php for CentOS servers
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"
  tags: [centos, apache]

- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb server (Ubuntu)
      apt:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "Ubuntu"
      tags: [ubuntu, db]

    - name: install mariadb server (CentOS)
      dnf:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "CentOS"
      tags: [centos, db]

- hosts: file_servers
  become: true
  tasks:

    - name: install samba package
      package:
        name: samba
        state: latest
      tags: samba
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml --list-tags site.yml
playbook: site.yml

play #1 (all): all    TAGS: []
TASK TAGS: [centos, ubuntu]

play #2 (web_servers): web_servers    TAGS: []
TASK TAGS: [apache, centos, ubuntu]

play #3 (db_servers): db_servers    TAGS: []
TASK TAGS: [centos, db, ubuntu]

play #4 (file_servers): file_servers    TAGS: []
TASK TAGS: [samba]
```

2.2

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml --tags centos -k -K site.yml
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]
ok: [192.168.56.104]
ok: [192.168.56.101]

TASK [install updates (CentOS)] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]
ok: [192.168.56.101]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
```

```
TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]

TASK [install mariadb server (CentOS)] *****
ok: [192.168.56.101]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]

PLAY RECAP *****
192.168.56.101      : ok=4   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignor
ed=0
192.168.56.103      : ok=3   changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignor
ed=0
192.168.56.104      : ok=2   changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignor
ed=0
```

2.3

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml --tags db -k -K site.yml
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]
ok: [192.168.56.101]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]

TASK [install mariadb server (Ubuntu)] *****
skipping: [192.168.56.101]

TASK [install mariadb server (CentOS)] *****
ok: [192.168.56.101]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]

PLAY RECAP *****
192.168.56.101      : ok=3   changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignor
ed=0
192.168.56.103      : ok=3   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignor
ed=0
192.168.56.104      : ok=2   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignor
ed=0
```

2.4

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml --tags apache -k -K site.yml
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]
ok: [192.168.56.101]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.103]
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]

PLAY RECAP *****
192.168.56.101      : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignor
ed=0
192.168.56.103      : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignor
ed=0
192.168.56.104      : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignor
ed=0
```

2.5

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml --tags "apache,db" -k -K site.y
ml
SSH password:
BECOME password[defaults to SSH password]:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]
ok: [192.168.56.101]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]

TASK [install mariadb server (Ubuntu)] *****
skipping: [192.168.56.101]

TASK [install mariadb server (CentOS)] *****
ok: [192.168.56.101]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]

PLAY RECAP *****
192.168.56.101      : ok=3   changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignor
ed=0
192.168.56.103      : ok=4   changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignor
ed=0
192.168.56.104      : ok=3   changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignor
ed=0
```

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

Figure 3.1.1

Make sure to save the file and exit.

You would also notice from our previous activity that we already created a module that runs a service.

Figure 3.1.2

This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently

running httpd using the command `sudo systemctl stop httpd`. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.

3. Go to the local machine and this time, run the `site.yml` file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.

To automatically enable the service every time we run the playbook, use the command `enabled: true` similar to Figure 7.1.2 and save the playbook

1

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```

```
GNU nano 7.2 site.yml *
- hosts: web_servers
  become: true
  tasks:

    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
      when: ansible_distribution == "Ubuntu"
      tags: [ubuntu, apache]

    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
      tags: [centos, apache]

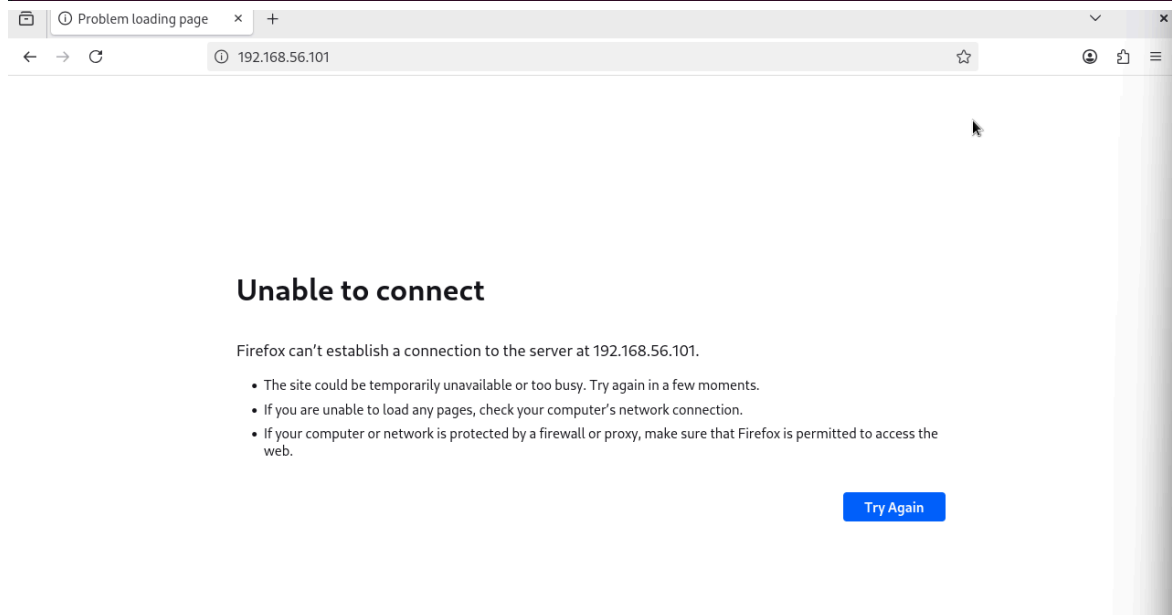
    - name: start and enable httpd service on CentOS
      service:
        name: httpd
        state: started
        enabled: yes
      when: ansible_distribution == "CentOS"
```

2

```
jerald@workstation:~/CPE212_Jerald_Interno$ ssh jeraldinterno@192.168.56.101
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Sat Sep  5 22:25:08 2026 from 192.168.56.1
[jeraldinterno@localhost ~]$ sudo systemctl stop httpd
[sudo] password for jeraldinterno:
[jeraldinterno@localhost ~]$ systemctl status httpd
○ httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
           └─php-fpm.conf
   Active: inactive (dead) since Sat 2026-09-05 22:29:20 PST; 7s ago
 Duration: 34min 49.212s
   Docs: man:httpd.service(8)
 Process: 941 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
 Main PID: 941 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec:  0 B/sec"
    CPU: 3.106s

Sep 05 21:54:25 localhost.localdomain systemd[1]: Starting The Apache HTTP Server...
Sep 05 21:54:29 localhost.localdomain httpd[941]: AH00558: httpd: Could not reliably determine the server's f>
Sep 05 21:54:29 localhost.localdomain systemd[1]: Started The Apache HTTP Server.
Sep 05 21:54:29 localhost.localdomain httpd[941]: Server configured, listening on: port 80
Sep 05 22:29:18 localhost.localdomain systemd[1]: Stopping The Apache HTTP Server...
Sep 05 22:29:20 localhost.localdomain systemd[1]: httpd.service: Deactivated successfully.
Sep 05 22:29:20 localhost.localdomain systemd[1]: Stopped The Apache HTTP Server.
Sep 05 22:29:20 localhost.localdomain systemd[1]: httpd.service: Consumed 3.106s CPU time.
lines 1-20/20 (END)
```



```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml site.yaml -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [web_servers] *****

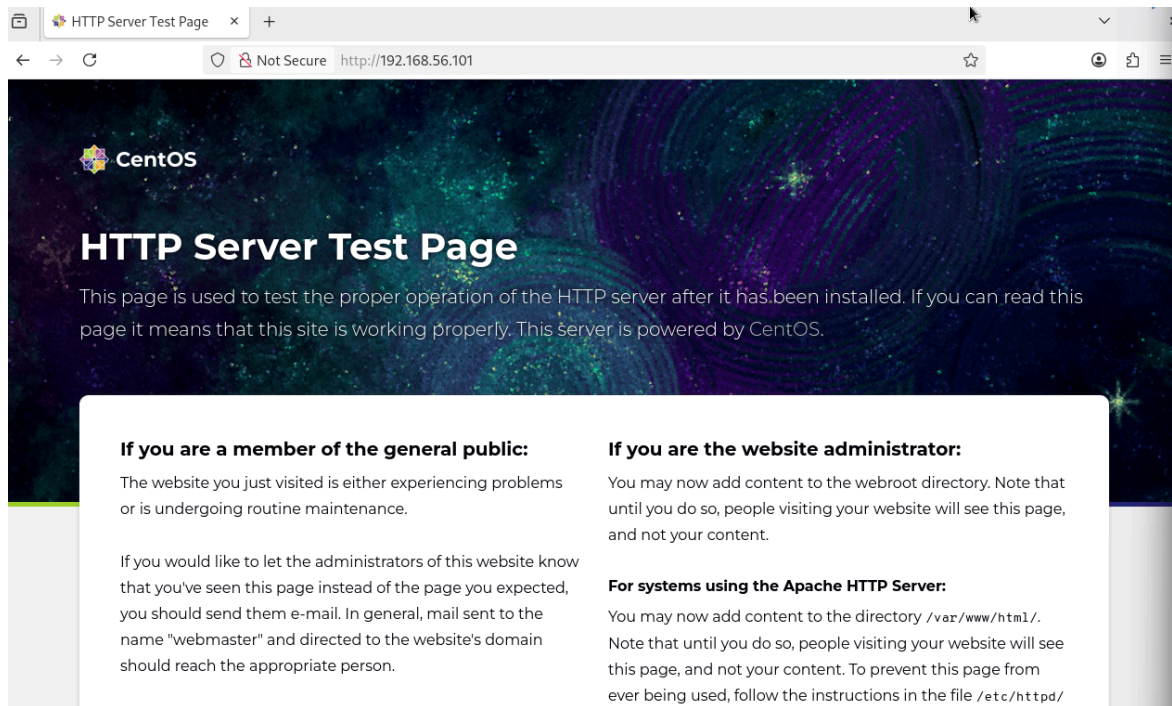
TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [install apache and php for Ubuntu servers] *****
ok: [192.168.56.103]
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]

TASK [start and enable httpd service on CentOS] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]

PLAY RECAP *****
192.168.56.103      : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.104      : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

1. Creating a Role for Web Servers

```
jerald@workstation:~/CPE212_Jerald_Interno$ mkdir -p roles/webserver/tasks roles/dbserver/tasks roles/fileserv  
er/tasks
```

2. Creating a Role for Web Servers

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano roles/webserver/tasks/main.yml
```

```
GNU nano 7.2      roles/webserver/tasks/main.yml *  
  
---  
- name: install apache and php (Ubuntu)  
  apt:  
    name:  
      - apache2  
      - libapache2-mod-php  
    state: latest  
  when: ansible_distribution == "Ubuntu"  
  
- name: install apache and php (CentOS)  
  dnf:  
    name:  
      - httpd  
      - php  
    state: latest  
  when: ansible_distribution == "CentOS"  
  
- name: start and enable httpd service (CentOS)  
  service:  
    name: httpd  
    state: started  
    enabled: yes  
  when: ansible_distribution == "CentOS"
```

3. Creating a Role for Database Servers

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano roles/dbserver/tasks/main.yml
```

```
GNU nano 7.2 roles/dbserver/tasks/main.yml *
---
- name: install mariadb server (Ubuntu)
  apt:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "Ubuntu"
- name: install mariadb server (CentOS)
  dnf:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano roles/fileserver/tasks/main.yml
```

```
GNU nano 7.2 roles/fileserver/tasks/main.yml *
---
- name: install samba package
  package:
    name: samba
    state: latest
```

4. Combining Multiple Roles in a Playbook

To orchestrate multiple roles in a single playbook for different server groups.

Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for Database.

```
jerald@workstation:~/CPE212_Jerald_Interno$ nano site.yml
```

```
GNU nano 7.2 site.yml
---
- hosts: web_servers
  become: true
  roles:
    - { role: webserver, tags: ['web', 'apache'] }
- hosts: db_servers
  become: true
  roles:
    - { role: dbserver, tags: ['db', 'mariadb'] }
- hosts: file_servers
  become: true
  roles:
    - { role: fileserver, tags: ['file', 'samba'] }
```

```
jerald@workstation:~/CPE212_Jerald_Interno$ ansible-playbook -i inventory.yaml site.yml -k -K
SSH password:
BECOME password[defaults to SSH password]:

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]
ok: [192.168.56.104]
ok: [192.168.56.101]

TASK [webserver : install apache and php (Ubuntu)] *****
skipping: [192.168.56.101]
ok: [192.168.56.103]
ok: [192.168.56.104]

TASK [webserver : install apache and php (CentOS)] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]
ok: [192.168.56.101]

TASK [webserver : start and enable httpd service (CentOS)] *****
skipping: [192.168.56.103]
skipping: [192.168.56.104]
ok: [192.168.56.101]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]

TASK [dbserver : install mariadb server (Ubuntu)] *****
skipping: [192.168.56.101]

TASK [dbserver : install mariadb server (CentOS)] *****
ok: [192.168.56.101]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.103]

TASK [fileserv : install samba package] *****
ok: [192.168.56.103]

PLAY RECAP *****
192.168.56.101      : ok=5    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.103      : ok=4    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.104      : ok=2    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?
 - Server grouping in the inventory file enables targeted orchestration across infrastructure layers. Instead of writing separate playbooks for individual machines, groups allow administrators to apply specific configurations (such as web, database, or storage tasks) to entire host categories simultaneously. This ensures consistency across environments and scales provisioning effortlessly as nodes are added.
2. What is the importance of tags in playbooks?
 - Tags provide granular execution control within large playbooks. By assigning tags to specific tasks or roles, administrators can execute or skip targeted operations (like., re-running only database configuration via `--tags db`) without executing the entire playbook. This reduces execution time, prevents unnecessary state checks on unaffected services, and simplifies troubleshooting.

3. Why do you think some services need to be managed automatically in playbooks?

- Many Enterprise Linux distributions (such as CentOS/RHEL) do not automatically enable or start daemon processes upon package installation. Automating service control using Ansible (state: started, enabled: yes) guarantees that necessary applications are running immediately after installation and will automatically recover following system reboots, ensuring high availability and eliminating manual intervention.

Conclusions/Learnings:

- Throughout this lab, key Ansible automation principles were implemented, including multi-play structure execution, dynamic OS-aware package provisioning, inventory host grouping, service lifecycle management, and role modularization. By utilizing Ansible tags, playbook execution efficiency was improved by isolating specific operational tasks. Furthermore, implementing systemd service management verified Ansible's self-healing capabilities, restoring disrupted services automatically and ensuring boot persistence. Converting monolithic plays into reusable Ansible Roles reinforced industry-standard configuration management practices, ensuring maintainable, modular, and scalable infrastructure automation.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: code assistance and explanation of concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.